OSSP • OPERATING SYSTEMS PROJECT

INTER-PROCESS CHAT APPLICATION

Using Inter-Process Communication (IPC)

SUBJECT

OSSP

PROJECT TEAM :

Harshitha.B — 2520080007
Krishnasree — 2520080009
Vennala — 252008018

Two independent processes. One conversation.

Introduction

A simple chat application that demonstrates how independent processes exchange information.

What is IPC?

- Inter-Process Communication allows separate processes to exchange data.
- Each process has its own memory space, so communication needs an OS-supported mechanism.
- IPC is used in operating systems, background services, and multi-process applications.

Our project idea

- Create two independent chat processes: User A and User B.
- Send and receive messages in both directions through named pipes (FIFOs).
- Display messages in the terminal and handle exit and common errors safely.

CORE IDEA

Process A ⇄ IPC channel ⇄ Process B

Problem Statement

Independent processes need a controlled way to exchange messages.

THE CHALLENGE

- Processes normally operate in separate memory spaces.
- One process cannot directly read another process's private memory.
- Communication must account for synchronization, message ordering, and process exit.
- A simple, demonstrable communication channel is needed for two-way exchange.

PROPOSED SOLUTION

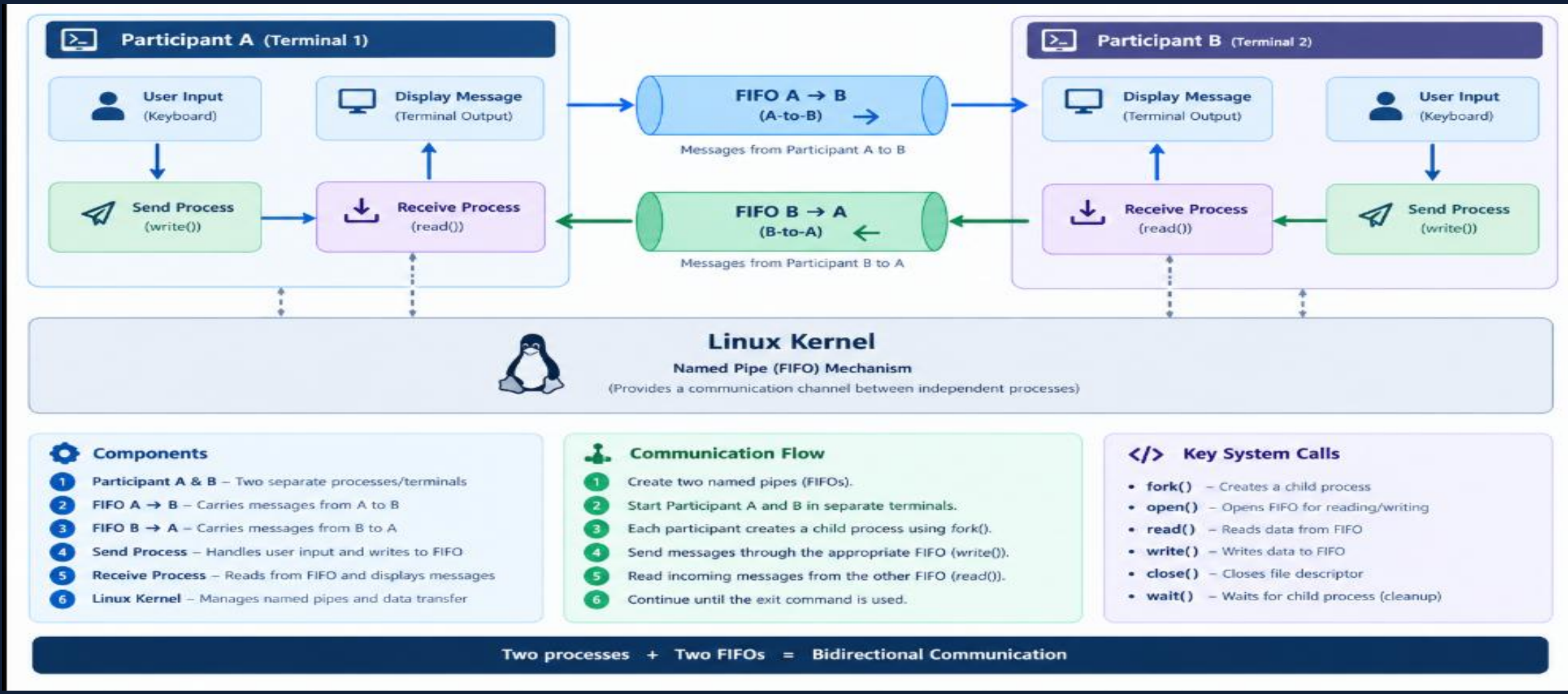
Build a two-way chat application using IPC.

Two named pipes provide separate paths for A → B and B → A.

Goal: reliable local message exchange with clean shutdown.

System Design

Two independent processes communicate through two one-way FIFO channels.



Implementation Approach

A practical sequence for building the prototype in C on Linux.

01 **Create FIFOs** — Create two named pipes for opposite message directions.

02 **Launch processes** — Run the same program as User A and User B in two terminals.

03 **Send & receive** — Use open(), read(), and write() to transfer messages.

04 **Handle concurrency** — Use a receiver thread or event-driven I/O so messages arrive while typing.

05 **Validate & exit** — Check input, handle unavailable peers, and close pipes cleanly.

Example exit command: /exit

Technologies Used

A lightweight stack chosen to highlight operating-system concepts.

C

CORE LANGUAGE

Implements process logic and IPC system calls.

Linux / Ubuntu

OPERATING SYSTEM

Supports POSIX APIs and named pipes.

POSIX FIFO APIs

IPC MECHANISM

mkfifo(), open(), read(), and write().

Pthreads

CONCURRENCY

Allows receiving and sending to proceed concurrently.

GCC

COMPILER

Builds the C program from source code.

VS Code + Terminal

DEVELOPMENT TOOLS

Edit, compile, run, and test two process instances.

Why this stack? It is low-cost, easy to demonstrate, and directly connected to OS-level IPC.

Testing

Test normal communication as well as error and shutdown scenarios.

TEST CASE	PROCEDURE	EXPECTED BEHAVIOUR
Basic messaging	A sends a message to B	B displays the correct message
Reverse reply	B sends a response to A	A receives the reply
Multiple messages	Send several messages in sequence	Messages appear in the intended order
Peer unavailable	Start only one process	App handles waiting or open failure safely
Exit handling	Enter /exit	Process closes resources cleanly

Testing note: record actual outcomes and screenshots after running the implementation.

Results & Demonstration

What the completed prototype should demonstrate.

```
krishnasree08@krishnasree08:~/ipc-chat$ cd ~/ipc-chat
./chat A

==================================
       IPC CHAT - USER A
==================================
Waiting for User B...

Connected! Chat started.
Type /quit to exit.

> Hello User B!
>
[14:48:41] User B: Hello User A!
>
[15:13:28] User B: HEY user A
>
[15:13:31] User B: hello
>
[15:13:33] User B: watsapp
> hello user b
> heyy
> wassupp
> ate?
```

```
krishnasree08@krishnasree08:~$ cd ~/ipc-chat
krishnasree08@krishnasree08:~/ipc-chat$ ./chat B

==================================
       IPC CHAT - USER B
==================================
Waiting for User A...

Connected! Chat started.
Type /quit to exit.

>
[14:48:17] User A: Hello User B!
> Hello User A!
> HEY user A
> hello
> watsapp
>
[15:13:45] User A: hello user b
>
[15:13:47] User A: heyy
>
[15:13:49] User A: wassupp
>
[15:13:52] User A: ate?
>
```

Expected outcomes

- Messages travel in both directions.
- Sender and message text are displayed clearly.
- Several messages can be exchanged in one session.
- The application handles exit and common errors

Illustrative output — replace with real terminal screenshots after testing.

Impact & Conclusion

A compact project that turns an OS concept into a working demonstration.

Impact & applications

- Builds practical understanding of process isolation and IPC.
- Relevant to local background services and separate software modules.
- Can be extended to multiple processes, a GUI, and message history.
- Security enhancements could add authentication and encryption.

Conclusion

- The design uses two independent processes and two FIFO channels.
- It demonstrates message transfer, concurrency, and cleanup.
- It provides a foundation for comparing FIFOs with queues and sockets.
- The prototype is local IPC, not an internet chat service.

KEY TAKEAWAY

IPC enables independent processes to coordinate without sharing private memory directly.

THANK YOU

Inter-Process Chat Application Using IPC

OSSP • Harshitha • Krishnasree • Vennala

Questions & Discussion